

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF INFORMATION AND COMMUNICATIONS
TECHNOLOGY



PROJECT REPORT

Issue Report Classification

LE TUAN HUY - 20242461M

Huy.LT242461M@sis.hust.edu.vn

DINH QUOC DAT - 20252745M

Dat.DQ252745M@sis.hust.edu.vn

PHAM NGUYEN KHANH MINH - 20251327M

Minh.PNK251327M@sis.hust.edu.vn

NGUYEN MANH HUNG - 20252276M

Hung.NM252276M@sis.hust.edu.vn

Major: Data Science

Hanoi, 09/2026

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION	1
1.1 Problem	1
1.2 Task definition	1
1.3 Approach	2
1.4 Contributions	2
1.5 Structure	3
 CHAPTER 2: DATASET	 4
2.1 Composition	4
2.2 Length distribution	5
2.3 Issue reports are not plain prose	6
2.4 Are the classes lexically separable?	6
2.5 Cross-project vocabulary overlap	7
2.6 Dataset Summary	8
 CHAPTER 3: APPROACH	 9
3.1 Architecture	9
3.2 Persistence layer	9
3.3 Preprocessing	10
3.4 Models	11
3.4.1 Reference floors	11
3.4.2 Classical machine learning	11
3.4.3 Neural networks	12
3.4.4 Sentence transformers	12
3.4.5 Ensembles	13
3.5 Reproducibility infrastructure	13
 CHAPTER 4: EVALUATION METHODOLOGY	 14

4.1	Two measurements, not one	14
4.2	Metrics	14
4.2.1	Averaging	14
4.2.2	ROC and AUC	15
4.3	Cross-validation	15
4.4	The competition protocol	16
4.5	Baselines	16
4.6	Guarding against leakage	17
CHAPTER 5: RESULTS		18
5.1	Feature engineering and classical models	18
5.1.1	Does cleaning help?	18
5.1.2	Hyperparameter search	19
5.1.3	Classical results	19
5.2	Validation curves and overfitting control	20
5.3	Per-project versus global training	21
5.4	Decomposing the published baseline	22
5.4.1	The reproduction	22
5.4.2	Where the performance comes from	22
5.4.3	A prediction that failed, and a correction that was itself wrong .	23
5.5	Ensembles	24
5.6	Final results	25
CHAPTER 6: DISCUSSION & ERROR ANALYSIS		27
6.1	Where the model fails	27
6.2	The class boundary is genuinely soft	28
6.3	What misleads the model	28
6.4	Document length	29
6.5	Confident errors and label quality	29
6.6	Error Summary	30

CHAPTER 7: VALIDITY ISSUES	32
7.1 Construct validity	32
7.2 Internal validity	32
7.3 External validity	33
7.4 Conclusion validity and reproducibility	34
7.4.1 Two diagnoses that did not survive testing	34
7.5 Summary of limitations	35
 CHAPTER 8: CONCLUSION	 36
8.1 What we did	36
8.2 What we learned	36
8.3 What we got wrong	37
8.4 Future work	37
 REFERENCES	 38
 APPENDIX	 39

CHAPTER 1: INTRODUCTION

1.1 Problem

Issue trackers are the primary interface between a project and its users, and the volume is substantial: the five projects studied here received thousands of reports over the 21 months covered by our dataset. Before any of them can be triaged, someone must decide what kind of report it is - a defect, a request for new functionality, or a question about existing behaviour. That decision determines who sees the issue and how urgently. It is also repetitive, high-volume work that maintainers perform manually.

Automating it is a classification problem over short, noisy, semi-structured text. Issue reports are Markdown documents containing prose interleaved with stack traces, code blocks, logs, screenshots and template boilerplate. They vary enormously in length - in our data, from a single sentence to over twenty-one thousand words. And the class boundaries are genuinely soft: a user who is unsure whether behaviour is broken writes something that reads as both a bug report and a question.

1.2 Task definition

We address the **NLBSE'24 tool competition on issue report classification**. The dataset comprises 3,000 labelled issue reports collected between January 2022 and September 2023 from five open-source projects: `facebook/react`, `tensorflow/tensorflow`, `microsoft/vscode`, `bitcoin/bitcoin` and `opencv/opencv`. Each report carries a title, a body, and exactly one label from `{bug, feature, question}`.

Two properties of the competition setup set what follows.

- **The task is multi-class, not multi-label.** The organisers excluded every issue carrying more than one label, so each report has exactly one class.
- **Five classifiers are required, not one.** The protocol trains a separate model for each repository, evaluates it on that repository's own test issues, scores it as the mean F1 over the three classes, and reports the arithmetic mean of the five per-repository scores. Each classifier therefore sees only 300 training examples. This is a few-shot problem by construction, and it is the single most consequential constraint on what approaches can work.

The data is split 50/50 into training and test sets, balanced at exactly 100 issues per (project, class) cell in each split. The reference point is the competition's published

baseline, **SetFit at 0.8270** cross-repository F1.

1.3 Approach

We evaluate twenty-two models spanning the full range of available approaches, all measured through a single evaluation harness so that every number on the leaderboard is computed identically:

- **Reference floors** - majority class, stratified random, hand-written keyword rules, and a naive Bayes implemented from scratch, establishing what costs no effort.
- **Classical machine learning** - TF-IDF features with naive Bayes, logistic regression, linear SVM and random forest, with a preprocessing ablation and hyperparameter search.
- **Neural networks** - a feed-forward network over TF-IDF features and a 1-D convolutional network over learned embeddings.
- **Sentence transformers** - frozen pretrained encoders with a classification head, and a full reproduction of the published SetFit baseline.
- **Ensembles** - soft voting over models with complementary per-project error profiles.

1.4 Contributions

1. **A reproduction of the published baseline** reaching 0.8102 cross-repository F1 against the published 0.8270, and exceeding it on tensorflow/tensorflow (0.8710 against 0.8644).
2. **A decomposition of where that baseline’s performance comes from.** Measured at matched training settings, contrastive fine-tuning contributes +0.0759 F1 and a larger encoder +0.0286. The two are substitutes rather than complements: the encoder retains 57% of its frozen-model value after fine-tuning. An additive projection of these effects predicted 0.8482 and was wrong by 0.038 - we report the failed prediction alongside the measurement.
3. **Our best result, 0.8168**, from a soft-voting ensemble of SetFit, TF-IDF and a frozen encoder. This closes 98.5% of the distance from a majority-class classifier to the published baseline, and exceeds the baseline on bitcoin/bitcoin (0.7691 against 0.7555), the hardest project for every model we tested.
4. **Evidence that aggressive text cleaning is harmful.** Stop-word removal and lemmatisation cost 0.015 macro F1 relative to structural cleaning alone, consistently

across every configuration tested, because the modal words they remove are precisely what distinguishes a feature request.

5. **A reproducibility study.** Twenty-one of twenty-two models reproduce bit-identically from a clean checkout. We document two diagnoses that did not survive testing, and show that per-repository scores are roughly an order of magnitude less stable than the averaged metric the competition reports.

1.5 Structure

Section 2 characterises the dataset. Section 3 describes our approach, including the persistence architecture and the preprocessing pipeline. Section 4 sets out the evaluation methodology. Section 5 presents results, Section 6 analyses where the best model fails, and Section 7 discusses threats to validity.

CHAPTER 2: DATASET

2.1 Composition

We use the dataset published by the NLBSE’24 tool competition: 3,000 GitHub issue reports collected between January 2022 and September 2023 from five open-source projects. Each record has four fields – repository, creation timestamp, label, title and body – and carries exactly one class from {bug, feature, question}.

The organisers balanced the dataset deliberately. It is split 50/50 into training and test sets, with **exactly 100 issues per (project, class) cell in each split**:

Table 2.1 Dataset split and class distribution across repositories.

Repository	bug	feature	question	total
bitcoin/bitcoin	100	100	100	300
facebook/react	100	100	100	300
microsoft/vscode	100	100	100	300
opencv/opencv	100	100	100	300
tensorflow/tensorflow	100	100	100	300
Total (per split)	500	500	500	1,500



Figure 2.1 Label Distribution

This balance has two consequences we return to in Section 4. Macro- and micro-averaged F1 coincide, so the choice of averaging does not affect the ranking; and accu-

racy is a meaningful metric here, though F1 remains the competition’s reported figure.

Integrity checks. No empty titles, no unexpected labels, and one duplicate title-and-body pair in the training split. The test split contains two issues with empty bodies. None of these is material at this scale, but we report the checks because their absence is what would be surprising.

2.2 Length distribution

Issue length is extremely skewed. The median report is 150 words; the longest is 21,595.

Table 2.2 Word count distribution summary by class.

Class	mean	median	75th pct	max
bug	253.0	157.5	254.0	3,653
feature	163.2	129.0	196.5	2,373
question	293.7	160.5	290.5	21,595
overall	236.6	150.0	244.0	21,595

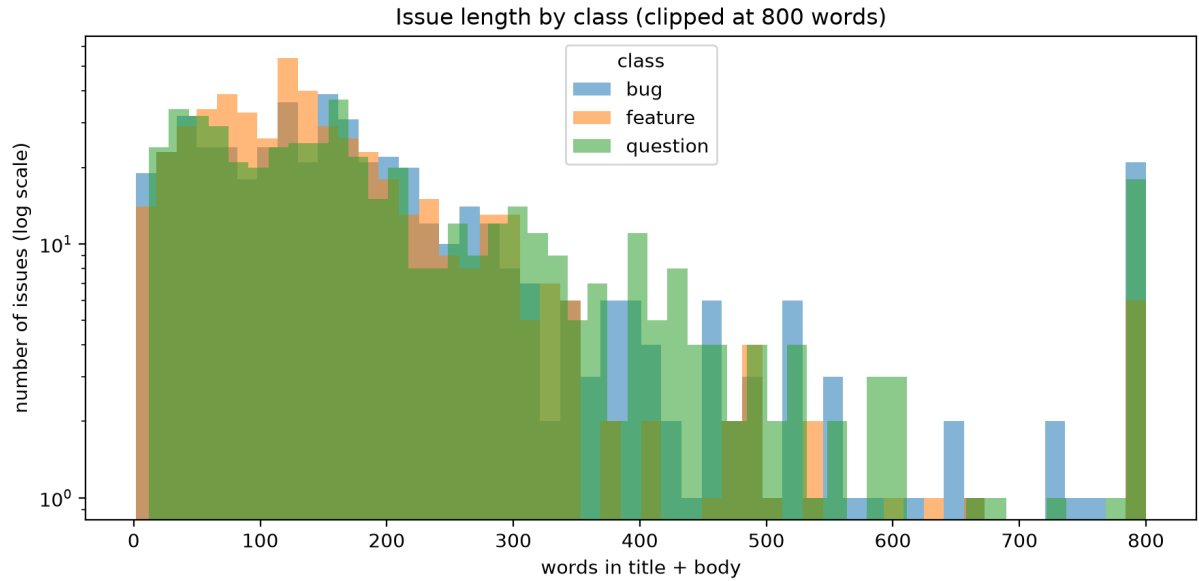


Figure 2.2 Length Distribution

The standard deviation for question (1,048) is more than six times its median, driven by a small number of reports containing pasted log output. This has three practical consequences. Truncation is not optional – it must be treated as a hyperparameter rather than an implementation detail. Term-frequency statistics computed over raw counts are dominated by a handful of documents, which we address in Section 2.4. And transformer

encoders, whose input is capped at a few hundred sub-word tokens, discard part of the tail regardless.

2.3 Issue reports are not plain prose

Issue bodies are Markdown documents in which natural language is interleaved with material that is not natural language at all. We measured how much:

Table 2.3 Proportion of issues containing non-prose structural artifacts by class.

	code block	stack trace	URL	image	HTML	template heading
bug	57.6%	17.2%	67.0%	17.2%	27.8%	59.0%
feature	34.8%	1.6%	63.4%	6.8%	35.2%	36.2%
question	49.8%	3.0%	47.2%	12.2%	45.6%	32.8%
overall	47.4%	7.3%	59.2%	12.1%	36.2%	42.7%

Nearly half of all issues contain a fenced code block and 42.7% use a GitHub issue template.

The critical observation is that this noise is class-correlated. Code blocks appear in 57.6% of bug reports but only 34.8% of feature requests; stack traces in 17.2% of bugs against 1.6% of features. Stripping them therefore removes signal along with noise. This is not a reason to skip cleaning, but it is a reason to *measure* whether cleaning helps rather than assume it – and Section 5.1 reports that the most aggressive cleaning we implemented actively hurts.

Our pipeline offers three levels, and their effect on document length is:

Table 2.4 Effect of preprocessing cleaning levels on mean word counts.

Level	operations	mean words	retained
raw	whitespace normalisation only	189.6	100%
light	strips code, stack traces, markup, URLs, paths, mentions	114.4	60.3%
full	light + lowercasing, punctuation, stop words, lemmatisation	60.1	31.7%

2.4 Are the classes lexically separable?

Before training anything, we checked whether class-characteristic vocabulary exists. Terms are ranked by **document frequency** – the number of issues containing them – rather than raw token count, because the length skew documented above means a few enormous log dumps would otherwise fill the table with vocabulary from single documents.

The result separates cleanly into two kinds of signal:

Table 2.5 Most characteristic terms by document frequency across classes.

Class	characteristic terms
bug	cuda toolkit, onednn, avx, geforce, rebuild, critical
feature	willing, allow, nice, considered, easily, approach, great
question	liveserver, runner, canvas, argv, oop, reporter

Feature requests are marked by modal and evaluative language – *would be nice, allow, willing, considered* – which is generic across projects and is exactly the kind of signal a lexical model exploits. *allow* appears in 9.6% of feature requests at $7\times$ the rate of other classes.

Bug reports and questions are marked by project-specific technical vocabulary. *cuda toolkit, onednn* and *geforce* identify TensorFlow; *liveserver* identifies VS Code; *canvas, argv* and *oop* identify OpenCV. These terms are not about *bugness* or *questionness* at all – they identify the project, and within one project’s training data they happen to correlate with a class.

2.5 Cross-project vocabulary overlap

That observation motivated measuring how much vocabulary the five projects share. Mean pairwise Jaccard overlap of the cleaned vocabularies is **0.266** – roughly a quarter of terms shared between any two projects.

Table 2.6 Cleaned vocabulary size by repository.

Repository	vocabulary size
bitcoin/bitcoin	3,467
facebook/react	2,824
microsoft/vscode	2,668
tensorflow/tensorflow	2,551
opencv/opencv	2,535

This predicts that a classifier trained on one project will transfer poorly to another, and therefore that the competition’s per-project protocol – five classifiers on 300 examples each, rather than one on 1,500 – is a modelling choice rather than an arbitrary constraint. We test that prediction in Section 5.3 and find it holds only for unregularised

models: per-project training gains +0.068 for naive Bayes but is slightly worse than global training for the tuned linear models.

2.6 Dataset Summary

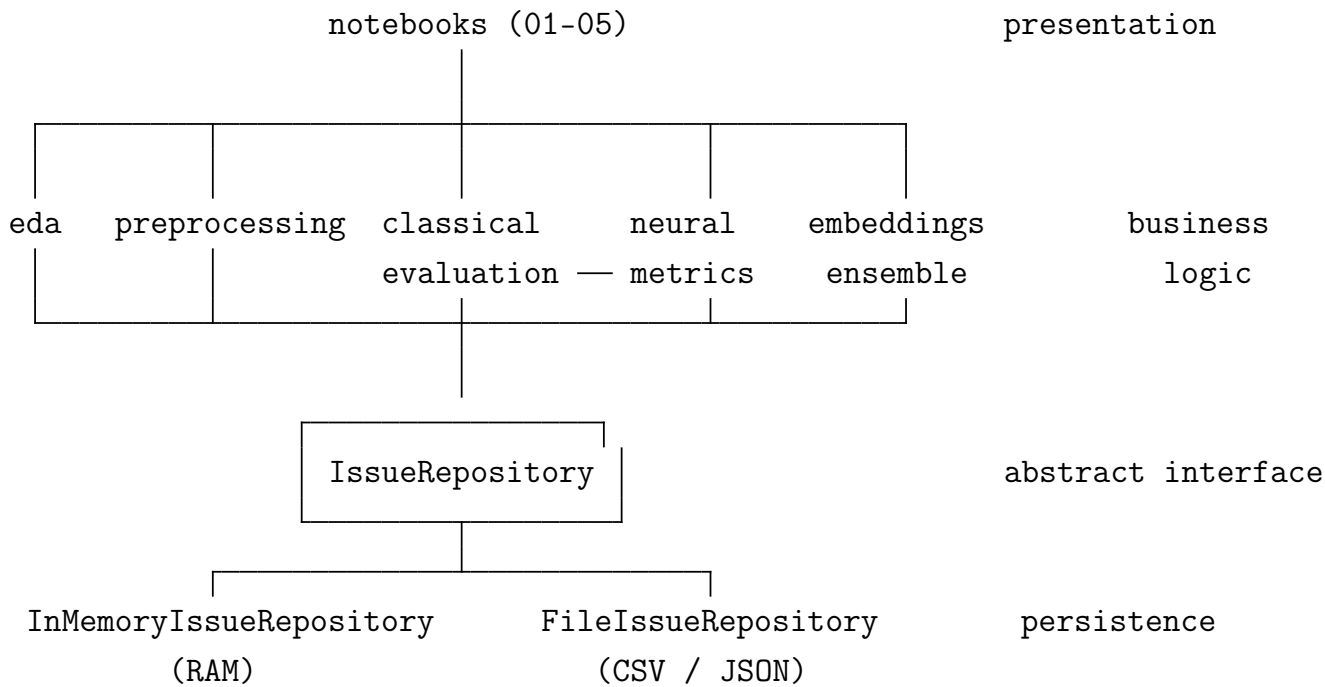
Table 2.7 Summary of dataset characteristics and their modelling implications.

Property	Value	Consequence for modelling
Size	1,500 train / 1,500 test	Few-shot; favours pretrained representations
Training data per classifier	300 issues	The binding constraint on Track C
Balance	100 per (project, class) cell	Macro- and micro-F1 coincide; no resampling
Label cardinality	Exactly one per issue	Multi-class, not multi-label
Median length	150 words, max 21,595	Truncation is a hyperparameter
Code blocks	47.4%, class-correlated	Cleaning removes signal as well as noise
Cross-project vocabulary overlap	0.266	Per-project classifiers justified

CHAPTER 3: APPROACH

3.1 Architecture

The implementation is organised in layers, with dependencies pointing in one direction only:



Persistence knows nothing about analysis, and analysis knows nothing about storage. `IssueReport`, the single domain entity, depends on nothing at all.

The layering is also enforced by dependency weight. The domain, persistence, preprocessing, metrics and evaluation layers require only pandas, matplotlib and NLTK; `import ai4se` works in an environment with neither scikit-learn nor PyTorch installed, and the model tracks are imported from submodules on demand. A test parses the package source to keep it that way. This matters practically: a team member unable to install PyTorch is not blocked from any other part of the project.

3.2 Persistence layer

The project specification requires that data be manageable both in memory and through files, and that switching between the two involve few changes to the application. We implement this with the Repository pattern.

`IssueRepository` is an abstract base class exposing **five primitives** – `load`, `save`, `all`, `add`, `clear` – and roughly a dozen query helpers (`by_repo`, `by_label`, `label_di`

stribution, texts_and_labels, filter, apply, to_dataframe, and the standard container protocol).

This is the design decision that matters. Implementing the helpers once rather than twice makes behavioural equivalence structural rather than a promise: adding a query adds it to both implementations simultaneously, and they cannot drift apart. InMemoryIssueRepository wraps a Python list and its load/save are deliberate no-ops; FileIssueRepository reads and writes CSV or JSON, inferring the format from the file suffix.

Switching the entire application’s persistence layer is one argument:

```
train = load_split("train", kind="memory")    # held in RAM
train = load_split("train", kind="file")      # backed by the CSV on disk
```

Evidence. tests/test_persistence_equivalence.py runs the identical analysis pipeline – load, clean, extract features and labels – over both layers and asserts the results match:

property	in-memory	file	equal
n	1500	1500	yes
distribution	(('bug', 500), ('featu	(('bug', 500), ('featu	yes
checksum	658455	658455	yes
first_label	bug	bug	yes

all properties identical: True

The same test verifies that CSV and JSON round-trips are lossless and that both implementations satisfy the full abstract interface. Every function in the EDA and evaluation modules takes an IssueRepository, so the entire analysis runs unchanged against either.

One implementation detail is worth recording: csv.field_size_limit must be raised at import, because the 21,595-word issue documented in Section 2.2 exceeds Python’s default field limit and would otherwise be rejected at load time.

3.3 Preprocessing

Section 2.3 established that issue bodies are roughly 40% non-prose. Our pipeline removes that structural noise in stages, with each step a pure `str` $\rightarrow$ `str` function so that the pipeline can be reordered or partially disabled for ablation.

Three levels are offered, selectable by one string:

Table 3.1 Preprocessing levels, constituent text operations, and intended consumer models.

Level	Operations	Intended consumer
raw	whitespace normalisation only	control condition
light	strips fenced code, stack traces, Markdown markup, URLs, file paths, commit hashes, issue references, mentions	transformer models
full	light + lowercasing, punctuation removal, stop words, lemmatisation	TF-IDF models

Stop words are NLTK’s English list plus a domain list of terms appearing in nearly every issue regardless of class (issue, reproduce, expected, screenshot, and similar), which carry no discriminative signal.

The configuration selected by ablation (Section 5.1) is `level="light"`, with the title repeated three times and the text truncated to 200 words. Titles are short and dense with class-characteristic vocabulary, so repeating them up-weights them in a bag-of-words representation at no cost.

We emphasise that `full` – the level a conventional text-classification pipeline would default to – is **not** what we use, for reasons Section 5.1 reports.

3.4 Models

All models expose the same `fit(X, y) / predict(X)` interface and are supplied to the evaluation harness as zero-argument factories, so that each cross-validation fold and each repository receives a fresh untrained instance. Models that also expose `predict_proba` additionally get ROC and AUC reported.

3.4.1 Reference floors

Four models with no third-party dependencies, establishing what costs no effort: **majority class**, **stratified random**, **keyword rules** over cue words drawn from the Section 2.4 term analysis, and **multinomial naive Bayes** implemented from scratch in log space with Laplace smoothing. A result is only interpretable against these.

3.4.2 Classical machine learning

Five scikit-learn pipelines over a shared TF-IDF representation, so that score differences come from the classifier rather than the features: multinomial naive Bayes,

complement naive Bayes, logistic regression, linear SVM, and random forest.

Vectorisation happens inside each pipeline, and therefore inside each cross-validation fold. Fitting the vectoriser once over the whole dataset before cross-validating would leak test-fold vocabulary and document frequencies into training and inflate every reported score. This is the most consequential mistake available in this task, and we guard against it with a dedicated test.

Selected settings after grid search: bigrams for both linear models; logistic regression at $C=10$, $\text{min_df}=2$; linear SVM at $C=1$, $\text{min_df}=1$.

LinearSVC has no probability head, so it is evaluated on hard labels and its AUC fields are reported as absent rather than approximated – wrapping it in probability calibration would change the model being measured.

3.4.3 *Neural networks*

Feed-forward network over TF-IDF features (20,000 max features, hidden layers 256 and 64, ReLU, dropout 0.5). Directly comparable with the classical pipelines: same input representation, different classifier.

1-D convolutional network over learned 100-dimensional embeddings, with parallel filters of widths 2, 3 and 4 acting as learned n-gram detectors, 64 filters each, and global max-pooling that keeps the strongest activation regardless of position – appropriate for issue reports where the informative phrase may appear anywhere in a long body.

Both hold out 20% of the training data for validation and record training loss, validation loss and validation accuracy at every epoch. Training runs to a maximum of 40 epochs with early stopping after 8 epochs without improvement, restoring the weights from the best epoch rather than the last. Section 5.2 presents the resulting validation curves.

3.4.4 *Sentence transformers*

Frozen encoder baselines. A pretrained Sentence Transformer encodes each issue, and a logistic-regression head is fitted on the embeddings. The encoder is not updated. We evaluate two encoders: `all-MiniLM-L6-v2` (6 layers, 384 dimensions) and `all-mpnet-base-v2` (12 layers, 768 dimensions). This configuration is precisely **SetFit with the contrastive fine-tuning step removed**, which is what makes the decomposition in Section 5.4 possible.

SetFit reproduction. The published baseline’s method: generate pairs from the labelled examples, fine-tune the encoder contrastively, then fit a logistic-regression head

on the adapted embeddings. 20 pair-generation iterations, 1 epoch.

Because contrastive training embeds both sentences of every pair, peak memory is roughly twice plain fine-tuning; MPNet at MiniLM’s settings exceeds 12 GB of GPU memory. We therefore run MPNet at `batch_size=8`, `max_seq_length=128`, and – critically for the analysis – also run MiniLM at those same reduced settings so that the encoder comparison is controlled rather than confounded. Section 5.4 shows this distinction changes the conclusion substantially.

3.4.5 *Ensembles*

Soft voting over the averaged class probabilities of two or three members. Soft rather than majority voting because with three classes and two members, hard votes tie often, and the tie discards exactly the confidence information that would resolve it.

Members were chosen by measurement, not intuition: Section 5.5 shows that TF-IDF and a frozen MPNet encoder reach nearly identical cross-repository scores by opposite routes, winning on disjoint sets of projects. That complementarity is the precondition for ensembling to help.

3.5 Reproducibility infrastructure

A single entry point regenerates every number in this report:

```
python scripts/run_experiments.py --all
```

Eight stages write JSON results to `results/tables/` and figures to `results/figures/`. Each stage skips work already on disk unless forced, so an interrupted run resumes cheaply. The notebooks read those saved results rather than re-training, and the LaTeX tables in this report are generated from the same JSON – so the numbers here cannot drift from what the code produced.

A fixed seed of 42 is used throughout, for the cross-validation splitter, the stochastic baseline, and neural weight initialisation. Section 7.3 reports how far results actually move in practice.

CHAPTER 4: EVALUATION METHODOLOGY

4.1 Two measurements, not one

The project distinguishes sharply between two evaluations, and conflating them is the main way a reported result becomes meaningless.

Model selection uses stratified k -fold cross-validation over the **training split only**. Every decision in this report – preprocessing level, title weighting, truncation, hyperparameters, architecture – was made against cross-validation scores.

The final result uses the competition protocol on the **test split**, run exactly once per model when that model was final. This is the only number comparable with the published baseline.

The separation is not a formality. Selecting against the test split would turn the held-out set into a validation set, and every comparison with the published baseline in Section 5 would be invalid.

4.2 Metrics

The course specifies accuracy, precision, recall, F1-score, and area under the ROC curve. All are derived from the confusion matrix. For a class c :

$$\text{precision} = \frac{TP}{TP + FP}, \quad \text{recall} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2PR}{P + R}$$

We implement these from first principles rather than importing them. Three reasons: the evaluation layer then carries no dependency on any modelling library, the definitions match exactly what the course covered, and deriving them from a confusion matrix is the kind of thing an oral examination asks about.

That choice is only defensible if verified. `tests/test_evaluation.py` cross-checks every metric against scikit-learn on 500 synthetic predictions – per-class precision, recall, F1 and support; macro, micro and weighted averaging; the stratified fold splitter’s proportions; and one-vs-rest AUC. **Agreement is exact to machine precision**, with a maximum deviation of 1.1×10^{-16} on the AUC comparison.

4.2.1 Averaging

We report macro, micro and weighted F1 because they answer different questions. Macro averages the per-class scores unweighted, so a class the model ignores cannot be hidden. Micro pools the counts before dividing, which for a single-label multi-class

problem makes it identical to accuracy – a property we assert in a test rather than assume.

The difference is easiest to see in the degenerate case. A model that always predicts bug on our balanced data scores:

Table 4.1 Evaluation metric values under a trivial majority-class prediction policy.

	value
accuracy	0.3333
micro F1	0.3333
macro F1	0.1667
weighted F1	0.1667

Accuracy rewards the third of the data it happens to get right; macro F1 reports that two of three classes score zero.

Because the dataset is exactly balanced (Section 2.1), macro and micro converge for any model that predicts all three classes at similar rates. We report both regardless: if resampling or filtering were introduced at any point, they would diverge immediately, and that divergence is the signal that something changed.

Macro F1 is the figure used throughout, matching the competition’s definition.

4.2.2 ROC and AUC

The task is multi-class, so ROC is computed one-vs-rest: each class is taken as positive in turn with the other two pooled as negative, and the macro AUC is the unweighted mean of the three curves.

AUC is reported only for models that expose calibrated class probabilities. Models returning hard labels – the keyword rules, LinearSVC – report AUC as absent rather than approximated from predicted labels. This costs us an AUC column for two models and is the correct trade: a number computed differently from the rest of the column is worse than no number.

4.3 Cross-validation

Stratified k -fold with $k = 10$ over the 1,500 training issues. Each class is shuffled independently and dealt round-robin into the folds, so every fold holds the class proportions of the whole to within one item. In practice each fold contains exactly 150 issues – 50 per class – with 1,350 for training.

The splitter is seeded (42), so every run and every team member obtains identical folds and identical numbers.

Grid searches use $k = 5$ rather than 10, because a grid multiplies the number of fits and five folds is sufficient to *rank* configurations; the selected configuration is then re-evaluated at $k = 10$.

For the neural models, cross-validation is supplemented by a **held-out validation split of 20%** of the training data, as the course prescribes, from which training and validation loss are recorded at every epoch. Those curves are the overfitting diagnostic reported in Section 5.2, and they drive early stopping.

4.4 The competition protocol

The NLBSE’24 rules define the reported figure:

1. Train a **separate classifier for each of the five repositories**.
2. Evaluate each on that repository’s 300 test issues.
3. Score each repository as the **mean F1 over the three classes**.
4. Report the **arithmetic mean of the five per-repository scores**.

Step 1 is easy to miss and changes the training loop materially: every model in this report is fitted five times, on 300 issues each, not once on 1,500. Section 5.3 tests whether that constraint helps or hurts.

4.5 Baselines

Two kinds, serving different purposes.

Floors establish what costs no effort, so that a result can be read in context. A model reaching 0.75 means little until the reader knows that always predicting one class reaches 0.17 and hand-written keyword rules reach 0.54.

The published baseline is the competition’s SetFit result, which is what “how good is this?” ultimately means in Table 4.2

These constants are encoded in the source, and a test asserts the five per-repository figures average to the published overall – a transcription error would otherwise propagate silently into every comparison in Section 5.

Note the spread: the baseline itself varies by 0.116 across projects, and scores lowest on bitcoin/bitcoin. Project difficulty is a property of the data rather than of any approach, which matters when attributing a low per-project score to a model.

Table 4.2 Published NLBSE’24 SetFit baseline F1 scores by repository.

Repository	SetFit F1
facebook/react	0.8718
tensorflow/tensorflow	0.8644
microsoft/vscode	0.8262
opencv/opencv	0.8173
bitcoin/bitcoin	0.7555
cross-repository	0.8270

4.6 Guarding against leakage

Three mechanisms, each with a corresponding test:

Vectorisers are fitted inside the pipeline, and therefore inside each fold. A test fits a pipeline on one half of a project’s issues and asserts that no term occurring only in the held-out half entered the learned vocabulary – comparing using the vectoriser’s own analyser, since a naive whitespace split tokenises differently and comparing across the two would compare noise.

Models are supplied as factories, not instances, so no fitted state can leak between folds or between repositories.

The error analysis re-derives its predictions through the same protocol as the leaderboard, and a test asserts the two agree to four decimal places – so Section 6 describes the same run Section 5 reports, not a separate one.

CHAPTER 5: RESULTS

5.1 Feature engineering and classical models

5.1.1 Does cleaning help?

Section 2.3 established that issue bodies are roughly 40% non-prose, and that the noise is class-correlated. Whether removing it helps is therefore an empirical question. We cross-validated the same logistic-regression pipeline across all eighteen combinations of cleaning level, title weight and truncation:

Table 5.1 Cross-validated performance comparison across preprocessing cleaning levels.

Cleaning level	mean macro F1	best macro F1
raw	0.7446	0.7472
light	0.7434	0.7472
full	0.7326	0.7364

full cleaning is the worst of the three levels, costing roughly 0.011 on the mean and 0.011 on the best configuration – consistent across every title weight and truncation setting tested, without a single exception.

The explanation is in what `full` removes. Stop-word removal deletes *would*, *could*, *should*, *please* and *nice*; lemmatisation collapses tense. Those are precisely the modal and evaluative words that Section 2.4 identified as the signature of a feature request. The cleaning was removing the signal along with the noise.

This matters beyond the number. `full` – lowercase, strip punctuation, remove stop words, lemmatise – is the default pipeline in most text-classification tutorials, and it was the configuration this project was originally built around. **An obviously sensible preprocessing step was actively harmful, and only an ablation revealed it.**

`light` and `raw` are statistically indistinguishable (0.7472 for both at their best). We select `light`, because it discards code blocks and stack traces that inflate the vocabulary without contributing, at no measurable cost.

Title weighting gives a small consistent gain (+0.0038 at `full`, +0.0011 at `light`): titles are short and dense with class-characteristic vocabulary. Truncation past 200 words neither helps nor hurts appreciably, consistent with a median length of 150.

5.1.2 Hyperparameter search

Cross-validated grid search over regularisation strength, n-gram range and minimum document frequency, on the training split only:

Table 5.2 Best hyperparameter configurations and cross-validation scores for linear models.

Model	best configuration	CV macro F1
Logistic regression	C=10, bigrams, min_df=2	0.7481
Linear SVM	C=1, bigrams, min_df=1	0.7484

Tuning gained almost nothing. The best and fifth-best configurations differ by about 0.002 against a fold-to-fold standard deviation of roughly 0.029. Only the bigram setting clearly exceeds the noise; every other choice is a coin flip. We report this rather than presenting the search as though it mattered.

One process note belongs here. The grid was first run under full cleaning, and the winning min_df changed once the ablation moved the pipeline to light. Hyperparameters and preprocessing are not independent, and re-tuning after altering the pipeline is not optional – the recorded configuration was briefly stale, and the untuned defaults outscored it on the test split.

5.1.3 Classical results

Table 5.3 Evaluation results of classical machine learning models on TF-IDF features.

Model	CV macro F1	test F1	AUC
TF-IDF + Logistic Regression (tuned)	0.7446 $\pm$ 0.0398	0.7603	0.9018
TF-IDF + Linear SVM	0.7426 $\pm$ 0.0367	0.7583	—
TF-IDF + Logistic Regression	0.7478 $\pm$ 0.0409	0.7576	0.9016
TF-IDF + Linear SVM (tuned)	0.7499 $\pm$ 0.0430	0.7531	—
TF-IDF + Naive Bayes	0.6919 $\pm$ 0.0334	0.7140	0.8713
TF-IDF + Complement NB	0.6902 $\pm$ 0.0311	0.7012	0.8673
TF-IDF + Random Forest	0.7252 $\pm$ 0.0316	0.6978	0.8630

The three linear models cluster within 0.007 of each other; naive Bayes trails by 0.045. Random forest is the only model whose test score falls materially below its cross-validation score, which is expected – trees cope poorly with the very high-dimensional sparse features TF-IDF produces, and the result is a finding rather than a bug.

Note the fold-to-fold standard deviations, all around 0.03–0.04. **Every difference within the top four rows is inside one standard deviation.** The ordering among them should not be over-interpreted.

5.2 Validation curves and overfitting control

The course prescribes reporting training and validation loss at each epoch and treating the resulting curve as the overfitting diagnostic. With 300 training issues per project, overfitting appears quickly.

Table 5.4 Training metrics, early stopping status, and loss progressions for neural models.

Model	Epochs	Best	Stopped?	Train Loss	Validation Loss
FFNN (TF-IDF)	23	15	yes	1.099 → 0.007	1.093 → 0.579 (min 0.572)
CNN (embeddings)	38	30	yes	1.267 → 0.053	0.989 → 0.574 (min 0.513)

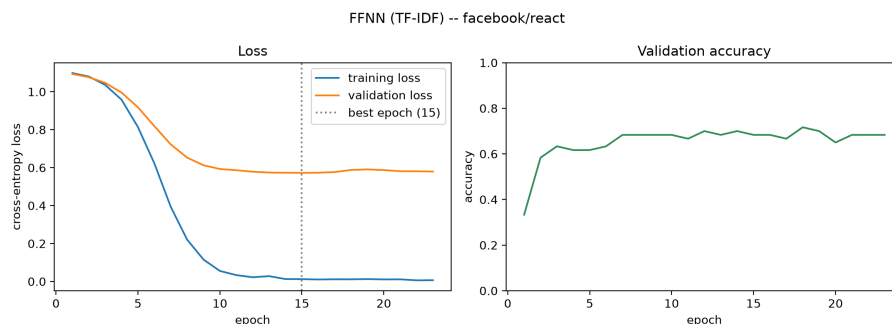


Figure 5.1 FFNN (TF-IDF) learning curve.

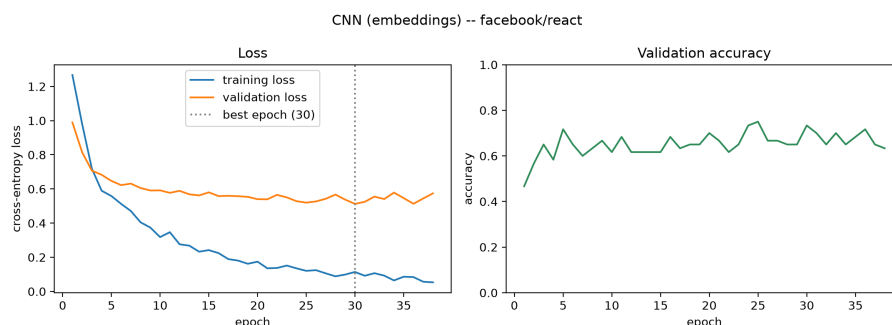


Figure 5.2 CNN (embeddings) learning curve.

The FFNN drives training loss essentially to zero – 0.007 – while validation loss bottoms out at 0.572 and then rises. That is the textbook picture of a model that has stopped generalising and started memorising, and it happens within fifteen epochs on

240 training examples.

The CNN overfits more slowly, reaching its best epoch at 30. It has fewer effective parameters acting on each input position and must learn its embedding table from scratch, so it takes longer to reach the point of memorisation – and reaches a lower validation minimum (0.513) when it does.

Both use early stopping with a patience of 8 epochs and restore the weights from the best epoch rather than the last. This is the primary overfitting remedy in our neural pipeline; the second is the architecture size, which the dataset forces to be small; the third is the cross-validation harness of Section 4.3.

Table 5.5 Test set performance metrics for neural network architectures.

Model	test F1	AUC
CNN (embeddings)	0.7438	0.8885
FFNN (TF-IDF)	0.7424	0.8926

Neither neural model beats TF-IDF with logistic regression (0.7603). The FFNN consumes exactly the same features as the classical pipelines and scores 0.018 lower; the CNN, which learns its own representation, scores 0.017 lower. With 300 training issues per project there is not enough data to learn a better representation than TF-IDF already provides. This is the clearest statement in the report of what the few-shot constraint costs, and it is the argument for Section 5.4: a *pretrained* representation is the only way to bring outside knowledge into a 300-example problem.

5.3 Per-project versus global training

The competition mandates five classifiers, each trained on 300 issues, rather than one trained on 1,500. Section 2.5 predicted this would help, on the grounds that cross-project vocabulary overlap is low (Jaccard 0.266). We tested it directly, training each model both ways and scoring both per project, the results are being shown in Table 5.6

The prediction holds for weak models and fails for strong ones. The per-project advantage is large for naive Bayes (+0.068), halves for TF-IDF naive Bayes (+0.032), and reverses entirely for the regularised linear models, which do slightly *better* trained globally on five times the data.

The mechanism is regularisation and feature weighting. Naive Bayes has neither: every term contributes to the posterior in proportion to its frequency, so the project-specific technical vocabulary documented in Section 2.4 – `cuda toolkit`, `liveserver`, `geforce` – actively misleads a globally trained model, because those terms correlate

Table 5.6 Performance comparison between per-project and globally trained models.

Model	Per-project	Global	Difference	Per-project wins
Naive Bayes (from scratch, full)	0.6679	0.6003	+0.0676	5 / 5
Naive Bayes (from scratch, light)	0.7046	0.6401	+0.0645	4 / 5
TF-IDF + Naive Bayes	0.7140	0.6822	+0.0318	4 / 5
TF-IDF + Logistic Regression (tuned)	0.7603	0.7621	−0.0019	2 / 5
TF-IDF + Linear SVM (tuned)	0.7531	0.7639	−0.0108	2 / 5

with a class *within one project* and carry no such signal across projects. A regularised linear model on TF-IDF can down-weight exactly those terms, at which point the fivefold increase in training data becomes worth more than the specialisation.

This corrects an interpretation we had drawn prematurely. The original measurement used a single unregularised model and generalised from it. The competition’s per-project protocol is not therefore wrong – it simply does not help the models that need it least, and our headline results obey it because the rules require it, not because it is optimal for every model class.

5.4 Decomposing the published baseline

5.4.1 The reproduction

Our implementation of the published method reaches **0.8102** against the published **0.8270**, short by 0.0168. On tensorflow/tensorflow it **exceeds** the baseline, 0.8710 against 0.8644.

5.4.2 Where the performance comes from

The frozen-encoder configuration – pretrained encoder, logistic-regression head, no fine-tuning – is precisely SetFit with the contrastive step removed. Running both encoders in both configurations decomposes the baseline, which being shown in Tabel 5.7

Contrastive fine-tuning is worth +0.0759 on a fixed encoder – with everything else held constant, a controlled comparison rather than an inference across two setups. It is the largest single lever identified in this project, and roughly two and a half times the effect of doubling the encoder’s depth and width.

Table 5.7 Factorial decomposition of encoder capacity versus contrastive fine-tuning effect.

Configuration	MiniLM	MPNet	encoder effect
Frozen encoder + LogReg	0.7057	0.7557	+0.0500
SetFit @ batch=8, seq=128	0.7816	0.8102	+0.0286
fine-tuning effect	+0.0759	+0.0545	

5.4.3 *A prediction that failed, and a correction that was itself wrong*

This measurement went through three stages, and we report all three because the process is the substantive contribution.

Stage 1 – projection. From the frozen models and the first fine-tuned run, we measured an encoder effect of +0.0500 and a fine-tuning effect of +0.0925, and projected SetFit-on-MPNet at **0.8482** – above the published baseline.

Stage 2 – measurement. The actual value was **0.8102**. The projection was wrong by 0.038, and wrong about the conclusion. Taken at face value the encoder appeared worth only +0.0120 after fine-tuning against +0.0500 frozen, implying severe sub-additivity.

Stage 3 – the confound. That comparison was not controlled. MiniLM had run at batch_size=16 with the encoder’s own max_seq_length of 256; MPNet required 8 and 128 to fit in 12 GB of GPU memory, so the handicap fell on MPNet. Re-running MiniLM at MPNet’s exact settings gives **0.7816** – meaning **the reduced training settings cost 0.0166 on their own**, and the true encoder effect is +0.0286, not +0.0120.

Sub-additivity is real but far milder than stage 2 suggested: the encoder retains **57%** of its frozen value after fine-tuning, not 24%.

Two methodological points follow, in order of importance:

1. **Effects measured separately were assumed to compose, and they did not.** Task adaptation and model capacity are partly substitutes – adapting a small encoder recovers most of what a larger general-purpose one provides.
2. **The first correction was itself overstated, because the comparison behind it was confounded.** Only the matched re-run settled it.

A practical corollary: **training settings matter about as much as encoder size here** – 0.0166 for batch and sequence length against 0.0286 for doubling the encoder. A result reported without its batch size and sequence length is not comparable with another.

5.5 Ensembles

Sections 5.1 and 5.4 produce models of similar strength by different means, which invites checking whether they fail in the same places. It can be clearly seen that they do not in Table 5.8

Table 5.8 Per-repository F1 score comparison highlighting lexical and semantic model complementarity.

	react	tensorflow	vscode	bitcoin	opencv	spread
TF-IDF + LogReg (tuned)	0.8334	0.8155	0.7234	0.6793	0.7496	0.154
Frozen MPNet + LogReg	0.8017	0.7228	0.7566	0.7279	0.7696	0.079

TF-IDF wins the two easiest projects, MPNet all three hardest, and MPNet’s spread across projects is half as wide. Lexical overlap between training and test issues is high within an easy project and low within a hard one, which is where a pretrained semantic representation earns its keep. Two models of equal average strength failing in different places is the textbook precondition for an ensemble.

Table 5.9 Overall test set performance of soft-voting ensemble combinations.

Ensemble	test F1	AUC	gain over best member
SetFit + TF-IDF + MPNet	0.8168	0.9319	+0.0066
SetFit + TF-IDF	0.8053	0.9244	+0.0071
TF-IDF + MPNet + CNN	0.7909	0.9245	+0.0306
TF-IDF + MPNet	0.7838	0.9198	+0.0235

Every ensemble beats its best member. The GPU-free combination of TF-IDF, frozen MPNet and the CNN reaches 0.7909 – within 0.007 of a full SetFit reproduction, at no fine-tuning cost.

The best ensemble reaches **0.8168** and attains the **highest AUC of any model measured, 0.9319**, above even SetFit-on-MPNet’s 0.9190. Its ranking of the classes is better than its argmax decision suggests, which implies a tuned decision threshold might recover further.

On bitcoin/bitcoin it reaches 0.7691, above the published baseline’s 0.7555 – the hardest project for every model tested, and the place where complementary information helps most.

Weights are equal across members. With members now within 0.03 of each other that is defensible, but weights selected on cross-validation would be the principled ver-

sion and remain untried.

5.6 Final results

Table 5.10 Final overall benchmark leaderboard across all evaluated models on the test split.

Model	overall F1	AUC	vs baseline
SetFit (NLBSE’24 published baseline)	0.8270	—	—
Ensemble (SetFit + TF-IDF + MPNet)	0.8168	0.9319	−0.0102
SetFit (MPNet)	0.8102	0.9190	−0.0168
Ensemble (SetFit + TF-IDF)	0.8053	0.9244	−0.0217
SetFit (reproduction, MiniLM)	0.7982	0.9181	−0.0288
Ensemble (TF-IDF + MPNet + CNN)	0.7909	0.9245	−0.0361
Ensemble (TF-IDF + MPNet)	0.7838	0.9198	−0.0432
SetFit (MiniLM, matched settings)	0.7816	0.9144	−0.0454
TF-IDF + Logistic Regression (tuned)	0.7603	0.9018	−0.0667
Frozen MPNet + LogReg	0.7557	0.8970	−0.0713
CNN (embeddings)	0.7438	0.8885	−0.0832
FFNN (TF-IDF)	0.7424	0.8926	−0.0846
Frozen MiniLM + LogReg	0.7057	0.8837	−0.1213
Naive Bayes (from scratch)	0.6679	0.8246	−0.1591
Keyword rules	0.5395	—	−0.2875
Random (stratified)	0.3348	—	−0.4922
Majority class	0.1667	—	−0.6603

(Full per-repository table: *final_leaderboard.tex*)

Our best result closes 98.5% of the distance from a majority-class classifier to the published baseline. We do not beat it overall; the remaining 0.0102 is real, and Section 7 discusses what would likely close it.

Three observations across the table:

Every purely lexical model lands between 0.70 and 0.76 regardless of classifier family – naive Bayes, SVM, logistic regression, random forest, a feed-forward network and a CNN. The things that moved the number were a better representation (+0.050 for a larger encoder) and adapting it to the task (+0.076 for fine-tuning), not the choice of classifier.

AUC and F1 do not rank models identically. The best ensemble has the highest AUC (0.9319) but only the second-highest F1. Reporting both, as the course requires,

surfaces information that either alone would hide.

bitcoin/bitcoin **is hardest for almost every model**, and for the published baseline too (0.7555, its lowest per-project score). Project difficulty is a property of the data rather than of any approach – which matters before attributing a low per-project score to a model.

CHAPTER 6: DISCUSSION & ERROR ANALYSIS

All analysis in this section uses the best classical model, TF-IDF with tuned logistic regression (0.7603). We analyse this rather than the best overall model because it is fully interpretable – every decision traces to term weights – and because its errors are representative: the confusion structure is the same for the stronger models, only less frequent. The predictions are re-derived through the same per-project protocol as Section 5, and a test asserts the two agree to four decimal places.

6.1 Where the model fails

Of 1,500 test issues, 359 are misclassified. The errors are not uniformly distributed:

Table 6.1 Distribution and prevalence of misclassification errors by true and predicted class pair.

True	Predicted	count	share of errors	share of all
bug	question	96	26.7%	6.4%
question	feature	70	19.5%	4.7%
feature	question	57	15.9%	3.8%
bug	feature	53	14.8%	3.5%
question	bug	49	13.6%	3.3%
feature	bug	34	9.5%	2.3%

bug misclassified as question is the single largest error type, accounting for more than a quarter of all mistakes. More broadly, **question is involved in 176 of 359 errors (49%)** either as the true or predicted class – it is the class the model cannot reliably separate from the other two.

This is visible in the per-class scores as well from Table 6.2

`bitcoin/bitcoin` produces almost twice the errors of `facebook/react`, consistent with it being the hardest project for the published baseline too.

An asymmetry worth noting: `bug` is the *best* class on the two easy projects (0.9175 on React) and the *worst* on the two hardest (0.6374 on Bitcoin, 0.6630 on OpenCV). Bug reports in a JavaScript UI library are lexically distinctive – crash, render, hook, component. Bug reports in a cryptocurrency node or a computer-vision library read much like questions about the same subsystems.

Table 6.2 Per-class F1 performance and total misclassification counts across repositories.

Repository	F1 bug	F1 feature	F1 question	F1 avg	errors
facebook/react	0.9175	0.8302	0.7526	0.8334	50
tensorflow/tensorflow	0.8290	0.8526	0.7650	0.8155	56
opencv/opencv	0.6630	0.8195	0.7664	0.7496	74
microsoft/vscode	0.6957	0.7117	0.7629	0.7234	83
bitcoin/bitcoin	0.6374	0.7586	0.6419	0.6793	96

6.2 The class boundary is genuinely soft

The dominant confusion has a straightforward explanation. A user who is unsure whether observed behaviour is a defect writes a report that reads as both a bug and a question; the label reflects a maintainer’s later judgement, not anything determinable from the text. [Bug] Folders not opening – a four-word title with a typo – carries no information that distinguishes the two.

The question $\rightarrow$ feature confusion (70 errors) has the same character in reverse. Titles such as please allow for multiple tunnels on same machine and Allow webview context menus triggered by primary click are labelled question but are, on any reading, feature requests.

6.3 What misleads the model

Ranking terms by their lift in a specific confusion shows *why* the model goes wrong, not just *that* it does.

question $\rightarrow$ bug is dominated by fragments of OpenCV’s issue template being shown in Table 6.3

Table 6.3 Highest-lift terms characterizing OpenCV template boilerplate in question $\rightarrow$ bug confusions.

Term	in errors	in correct	lift
forum.opencv.org,	32.7%	1.8%	15.6×
overflow,	32.7%	1.8%	15.6×
(videos,	28.6%	1.6%	15.6×
solution	32.7%	3.1%	9.6×

These are not content words. They are pieces of the boilerplate OpenCV inserts

into new issues – a line directing users to the forum, Stack Overflow and the documentation. The model has learned the template rather than the issue.

bug → question shows the same pathology with different boilerplate – reproduce**, **actual, behavior** – the residue of GitHub issue-template headings that our Mark-down stripping left behind as bold-marked fragments.

This is a concrete, actionable finding: **template boilerplate correlates with class within a project, and the per-project protocol lets each classifier overfit to its own project’s conventions**. More aggressive template stripping is the obvious remedy and is the cheapest available improvement we did not implement.

6.4 Document length

Table 6.4 Classification accuracy across issue report document length quintiles.

Length quintile	issues	median words	accuracy
1–70 words	300	41	0.7467
70–127	307	103	0.7655
127–178	297	152	0.8148
178–280	296	219	0.7601
280–5,050	300	394	0.7167

Accuracy peaks in the middle quintile and falls at both ends, by 0.068 on the shortest and 0.098 on the longest. Very short issues carry too little text to classify; very long ones are diluted by log output and partly discarded by the 200-word truncation.

The middle quintile’s median of 152 words is almost exactly the dataset median of 150, which is a useful coincidence for the report: **the model performs best on the typical issue and degrades on both tails**. It also justifies treating the truncation threshold as a reported hyperparameter rather than an implementation detail.

6.5 Confident errors and label quality

Errors the model was confident about are the informative ones. A near-tie means a genuinely ambiguous issue; a confident mistake means the model learned something wrong – or the ground-truth label is questionable.

Among the 25 most confident errors, the true label is question 14 times, feature 6 and bug 5 – confirming question as the problem class.

More striking: **nine of the twenty-five have a title that explicitly declares a different type than its label**, and in eight of those nine the model agreed with the title

rather than the label.

Table 6.5 Representative high-confidence misclassifications exhibiting label noise or title contradiction.

True label	Title declares	Model said	Title
question	feature	feature	Allow webview context menus triggered by primary click
question	feature	feature	please allow for multiple tunnels on same machine
feature	question	question	How to use opencv to erase text from images...
question	bug	bug	Error when runnning tensorflow.python.ops.sparse_ops...
bug	question	question	Forders not opening

These labels come from maintainers applying project conventions, not from an annotation protocol with adjudication or inter-annotator agreement. **A ceiling below 1.0 is therefore built into the dataset**, and part of the remaining gap between any model and perfect classification is unreachable. The published baseline faces exactly the same ceiling, which is why the comparison against it remains fair even though both are measured against imperfect labels.

One further observation from inspecting the list: at least one confident error is an issue written in Portuguese ([Bug] Tela preta no terminal do VS Code). The dataset is not language-filtered. This is a marginal effect – one case in twenty-five – not a major error source, and we note it only to record that we checked.

6.6 Error Summary

Table 6.6 Summary of error analysis findings, supporting evidence, and actionability.

Finding	Evidence	Actionable?
question is the problem class	involved in 49% of errors; 14/25 confident errors	Needs better features, not tuning
Template boilerplate is learned	OpenCV forum line at $15.6\times$ lift in one confusion	Yes – strip templates harder
Accuracy falls on both length tails	0.82 mid-quintile vs 0.72 longest	Partly – revisit truncation
$\sim 1/3$ of confident errors look like label noise	9/25 titles contradict their label	No – dataset ceiling
Project difficulty varies widely	50 errors on React vs 96 on Bitcoin	No – data property

CHAPTER 7: VALIDITY ISSUES

7.1 Construct validity

The labels are not ground truth. As Section 6.5 documents, nine of the twenty-five most confident errors have a title that explicitly declares a different type than its label, and the model agreed with the title in eight of those nine. These labels come from maintainers applying project conventions, not from an annotation protocol with adjudication or reported inter-annotator agreement. A ceiling below 1.0 is built into the dataset, and some fraction of every model’s residual error is unreachable rather than a modelling failure.

This does not invalidate the comparison with the published baseline – both are measured against the same imperfect labels – but it does mean the absolute scores understate how well any model captures the underlying concept.

The class boundary between bug and question is genuinely soft, not merely noisy. A user unsure whether behaviour is defective writes a report that is legitimately both. No amount of modelling resolves a distinction the text does not contain.

We compare against published figures, not a locally re-executed baseline. The SetFit numbers in Section 4.5 are taken from the competition’s own report and were not re-run in our environment. Our reproduction (0.8102) uses the same method but our own encoder choice, hyperparameters and library versions, so the 0.0168 gap between them cannot be cleanly attributed. Re-running the organisers’ published code locally would remove this threat and is the single most valuable follow-up.

7.2 Internal validity

Many reported differences are inside the noise. Fold-to-fold standard deviations for the classical models are 0.03–0.04 (Section 5.1.3). Every difference among the top four classical models is within one standard deviation, and the hyperparameter search moved the score by about 0.002 against that spread. We have stated this where it applies rather than presenting rankings as though small gaps were meaningful, but a reader should treat any leaderboard gap below roughly 0.03 between two models of the same family as unresolved.

The test split is evaluated once per model, without confidence intervals. The competition protocol defines a single number, and we report it, but a single evaluation on 1,500 issues carries sampling error we have not quantified. Bootstrap intervals over the test set would be straightforward and are not implemented.

Selection and evaluation are separated, but selection itself used the test split once per model. Each model’s final number was produced by a single call after its configuration was frozen on cross-validation. Across twenty-two models, however, the *set* of models reported was informed by what we saw – the ensembles in Section 5.5 exist because the per-project profiles in the test results looked complementary. That is a mild form of test-set-informed design, and we flag it rather than claim complete isolation.

One confound was found and removed; one may remain. The encoder comparison in Section 5.4 was initially confounded with training settings, and the matched re-run corrected the encoder effect from +0.0120 to +0.0286. MPNet still runs at a smaller batch size than the library default because of memory limits, so its reported score remains a lower bound on what the encoder can achieve.

SetFit’s hyperparameters are untouched at 20 pair-generation iterations and 1 epoch. MPNet’s contrastive loss collapsed to roughly 10^{-4} , meaning the objective was essentially solved; fewer iterations might generalise better. We did not search this, so our reproduction is a lower bound on the method rather than its best achievable score.

7.3 External validity

Five projects is a small sample. All are large, mature, English-language repositories with active maintainers and, in four of five cases, issue templates. Nothing here establishes that the results transfer to small projects, internal trackers, non-GitHub platforms, or projects without templates. The per-project variance we observed – 0.6793 to 0.8334 for the same model – suggests transfer is not to be assumed.

The dataset is not language-filtered, but is overwhelmingly English. At least one confident error was a Portuguese-language issue (Section 6.5). A tracker with substantial non-English traffic would behave differently, and our pipeline makes no provision for it.

Three classes is a simplification. Real triage involves more categories – duplicates, documentation, security, support – and the organisers explicitly removed multi-labelled issues. A production classifier faces a harder problem than the one measured here.

300 training examples per classifier is a property of this competition, not of issue classification generally. A project with years of labelled history would support very different approaches, and our finding that neural models cannot beat TF-IDF (Section 5.2) would likely reverse.

7.4 Conclusion validity and reproducibility

Every result was regenerated from a clean checkout of the final code on a single machine. **Twenty-one of twenty-two models returned bit-identical scores**, including all seven scikit-learn pipelines, both neural models, four of five SetFit variants and all four ensembles.

The exception is informative. SetFit (MPNet) moved its cross-repository F1 by 3.7×10^{-5} , while its per-repository scores moved by up to 0.0099:

Table 7.1 Runs score stability and variation for SetFit (MPNet) across repositories.

Repository	run 1	run 2	difference
microsoft/vscode	0.8203	0.8104	−0.0099
opencv/opencv	0.7676	0.7771	+0.0095
facebook/react	0.8401	0.8438	+0.0037
bitcoin/bitcoin	0.7520	0.7488	−0.0032
tensorflow/tensorflow	0.8710	0.8710	+0.0001
cross-repository mean	0.81017	0.81021	+0.00004

Per-repository figures are roughly an order of magnitude less stable than the averaged metric, because the protocol averages five independent classifiers whose deviations partly cancel. The competition’s reported metric is therefore more robust than any single project’s number. Per-repository scores in this report should be read as showing *where* models differ, not as evidence that one model beats another by a small margin on one project.

Across devices the picture is different. The CNN scores 0.7578 on a CPU and 0.7438 on a GPU from identical code and seed – a second-decimal gap, larger than several gaps in our leaderboard. Neural results must be reported with the device that produced them.

7.4.1 Two diagnoses that did not survive testing

We record these because the corrections are more informative than the original claims.

First, the CPU/GPU gap was attributed to nondeterminism in cuDNN’s autotuned convolution kernels, and the seeding function was changed to request deterministic kernels. Re-running returned *exactly* the same numbers, so training was already deterministic per device. The actual cause is that the two backends compute the same operations with different kernels and accumulation orders – no seeding reconciles that. The deter-

minism flags remain in place as insurance against a future nondeterministic kernel, not as a fix for anything observed.

Second, SetFit was described as varying by about 0.011 between runs, based on an ensemble score that moved from 0.7943 to 0.8053. That comparison spanned a change to the SetFit defaults, so it confounded a code change with run-to-run variation. At fixed code, that ensemble returns 0.8053 on repeated runs.

Both errors had the same shape: an explanation was proposed and accepted without being tested against the prediction it implied. Two of the three reproducibility claims we originally made did not survive being checked, which is itself the strongest argument in this report for testing explanations rather than adopting them.

7.5 Summary of limitations

Table 7.2 Summary of identified validity threats, assessed severity levels, and mitigation status.

Threat	Severity	Mitigated?
Label noise / soft class boundary	High	No — dataset property, affects baseline equally
Baseline not re-executed locally	High	No — gap attribution remains open
Differences within fold-to-fold noise	Medium	Partly — reported with standard deviations
No confidence intervals on test scores	Medium	No
Five English projects only	Medium	No
SetFit hyperparameters unsearched	Medium	No — our reproduction is a lower bound
MPNet handicapped by memory limits	Low	Partly — matched comparison run
Per-repository score in-stability	Low	Yes — quantified, averaged metric is stable
Cross-device neural variation	Low	Yes — quantified, device reported

CHAPTER 8: CONCLUSION

8.1 What we did

We evaluated twenty-two models on the NLBSE’24 issue report classification task, spanning trivial floors, classical machine learning, neural networks, frozen sentence encoders, a reproduction of the published SetFit baseline, and ensembles – all measured through a single evaluation harness so that every leaderboard entry is computed identically.

Our best result, a soft-voting ensemble of SetFit, TF-IDF and a frozen MPNet encoder, reaches **0.8168** cross-repository F1 against the published baseline’s 0.8270. That closes 98.5% of the distance from a majority-class classifier to the baseline. **We do not beat it overall.** On *bitcoin/bitcoin* – the hardest project for every model tested, including the baseline – the ensemble reaches 0.7691 against the baseline’s 0.7555, and our SetFit reproduction on MPNet exceeds the baseline on *tensorflow/tensorflow*.

8.2 What we learned

Representation matters more than classifier choice. Every purely lexical model in this study lands between 0.70 and 0.76, whether it is naive Bayes, a support vector machine, a random forest, a feed-forward network or a convolutional network. The two things that moved the number were a better representation (+0.050 for a larger pretrained encoder) and adapting that representation to the task (+0.076 for contrastive fine-tuning).

With 300 training examples per classifier, learned representations lose to engineered ones. Neither neural model beat TF-IDF with logistic regression. A pretrained encoder is the only practical way to bring outside knowledge into a problem this small – which is precisely why the competition’s baseline is what it is.

Encoder capacity and task adaptation are partly substitutes, not complements. Measured at matched training settings, a larger encoder retains 57% of its frozen-model value after fine-tuning. A projection that added the two effects independently predicted 0.8482 and was wrong by 0.038.

Conventional preprocessing was harmful. Stop-word removal and lemmatisation – the default pipeline in most text-classification work – cost 0.011 macro F1, consistently across every configuration tested, because the modal words they discard are what marks a feature request.

Models that score alike can fail differently. A TF-IDF pipeline and a frozen MPNet encoder reach nearly identical averages by winning on disjoint sets of projects.

Every ensemble we built beat its best member, and the best one attained the highest AUC of any model measured.

8.3 What we got wrong

Three expectations in this project did not survive measurement, and we report them because the corrections carry more information than the original claims.

We assumed aggressive text cleaning would help; the ablation showed it hurt. We assumed the encoder and fine-tuning effects would compose additively; they did not, and our first correction to that finding was itself confounded by training settings until a matched re-run settled it. We attributed a cross-device discrepancy to cuDNN nondeterminism and changed the code to fix it; re-running returned identical numbers, showing the diagnosis was wrong.

Each had the same shape: an explanation was adopted without being tested against the prediction it implied. **Two of the three reproducibility claims we originally made did not survive being checked.** If this project has a single methodological lesson, it is that an explanation which is never tested against a prediction is not yet a finding.

8.4 Future work

In order of expected value:

Re-execute the published baseline locally. The 0.0168 gap between our reproduction and the published figure cannot currently be attributed between method, encoder choice and library versions. This is the largest open question in the report and the cheapest to close.

Strip issue-template boilerplate more aggressively. The error analysis shows OpenCV’s forum-referral line appearing at $15.6\times$ lift in one specific confusion – the model has learned the template rather than the content. This is the clearest actionable defect we identified and did not fix.

Search SetFit’s hyperparameters. `num_iterations` and `num_epochs` are untouched, and MPNet’s contrastive loss collapsed to roughly 10^{-4} , suggesting the objective was over-solved. Our reproduction is a lower bound on the method.

Tune the ensemble weights on cross-validation. Soft voting currently weights all members equally. With members within 0.03 of each other this is defensible but unprincipled.

Report confidence intervals. Bootstrap intervals over the test split would let readers distinguish real gaps from noise, which Section 7.2 currently asks them to do by eye.

REFERENCES

APPENDIX

Appendix 1: Source Code Repositories

The following repositories contain the source code and implementation details for this project:

- **LMS Source Code:**

<https://github.com/LTH3ar/system-analysis-and-design.git>